PhishGuard: A Dynamic Phishing URL Detection System with Machine Learning and Crowdsourced Threat Intelligence

D.ANISH MEGHAN
Roll No. 23K91A6750
Department of Computer Science and Engineering (Data Science)
[COLLEGE NAME]
[CITY]
[EMAIL]

B.AKSHAYA
Roll No. 23K91A6737
Department of Computer Science and Engineering (Data Science)
[COLLEGE NAME]
[CITY]
[EMAIL]

B.TANUJA
Roll No. 23K91A6736
Department of Computer Science and Engineering (Data Science)
[COLLEGE NAME]
[CITY]
[EMAIL]

CH.SAI CHETHAN
Roll No. 23K91A6742
Department of Computer Science and Engineering (Data Science)
[COLLEGE NAME]
[CITY]
[EMAIL]

Mr. G. Ravi
[DESIGNATION], Project Guide
Department of Computer Science and Engineering (Data Science)
[COLLEGE NAME]
[CITY]
[EMAIL]

***Abstract*—**Phishing remains a leading route to credential theft, and blacklists and one-time-trained classifiers miss newly created URLs. This paper presents PhishGuard, a phishing URL detection system that follows the three-stage design of a recent base paper: feature extraction, machine-learning classification and crowdsourced threat-intelligence refinement. Forty-seven offline lexical, structural and domain features are extracted from the URL string alone, and Random Forest, XGBoost and Logistic Regression are compared on a balanced 40,000-URL sample. A community store confirms a URL after two net votes or one trusted-feed report; a confirmed verdict overrides the model at once and feeds a retraining loop with hot reload. On 8,000 held-out URLs XGBoost reaches accuracy 92.93 %, F1 92.88 % and ROC-AUC 0.981, with five-fold cross-validated F1 of 92.56 %; a check takes 8.0 ms. In a live run two reports turned a page the model had scored at 9.5 % into a phishing verdict, whereas retraining with random in-distribution URLs did not change the average score. The dataset is small and from one source, so the results are not comparable with the base paper's 99.8 %, and no user study was carried out.

***Index Terms*—**adaptive security, crowdsourced threat intelligence, machine learning, phishing detection, URL classification, XGBoost.

I. INTRODUCTION

Phishing remains one of the most common routes to stolen credentials and financial loss: an attacker imitates a trusted service and lures the victim to a look-alike page through a link [2]. The link is the one artefact every campaign needs, so judging it before the page is opened is attractive: a URL is short, always available and cheap to inspect. Today links are checked mostly by blacklists, fixed rules and classifiers trained once on a fixed dataset. Blacklists act only after a URL has been reported, so they miss zero-day sites whose domains may live only hours; fixed rules are easy to evade; and static models age as attack patterns change [1].

The base paper argues that these weaknesses are complementary: a classifier can score a URL it has never seen, while a crowd of users and feeds recognises a new campaign before a model could learn it [1]. It proposes a three-phase framework, extractor, detection and crowdsourced updating, but evaluates the crowd loop by simulation on one benchmark. This paper builds and measures a small, complete version of that idea. PhishGuard is a phishing URL detection system that combines offline URL features, three classifiers and a community threat-intelligence store whose confirmed verdicts override the model and retrain it. Our contributions are:

1) a 47-feature offline URL extractor (23 lexical, 9 structural, 15 domain) with human-readable red flags, and a fair comparison of Random Forest, XGBoost and Logistic Regression;
2) a vote-based community store with trusted public feeds in which a confirmed phishing or safe verdict overrides the model at once, and a retraining loop with hot reload;
3) a live end-to-end evaluation on real feed data, plus ablation, threshold and crowd-feedback experiments that state plainly what retraining does and does not change;
4) a Flask API and a seven-page React interface, with 95 automated checks and documented limits.

Section II reviews the literature, Section III describes the method, Section IV reports the results and Section V concludes.

A. System Hardware

All measurements were taken on one desktop with an Intel Core i5-12500 (6 cores, 12 threads), 15.7 GB of memory and Windows 11 Pro. No GPU was used. Training, evaluation and timing ran on this machine only; other hardware was not tested.

B. Software Requirements

The system uses Python 3.11.9, scikit-learn 1.4.2 [3], XGBoost 2.0.3 [4], pandas 2.2.1, NumPy 1.26.4 and joblib for the models, Flask 3.1 [5] with SQLite for the API and storage, and React 18.3 with Vite 5.4 [6] for the interface. Node.js 24 served the frontend. Everything runs offline except the optional feed synchronisation.

II. LITERATURE SURVEY

A. Base Paper

Aldhuwayhi and Alromih [1] describe a dynamic framework with three phases. The *extractor* derives features from the URL and answers directly if the URL matches a

known pattern. The *detection* phase classifies the remaining URLs with a trained model and flags inputs that match no known pattern as novel. In the *updating* phase novel URLs are queued for human verification, which filters noisy reports; verified URLs are added to the training set and the model is retrained periodically. They evaluate on PhiUSIIL [7], 235,804 URLs with 54 URL and page-content features, balance the classes by undersampling, keep 34 features after ANOVA and chi-square selection, and train a hybrid network: an embedding and an LSTM with attention over the URL text, concatenated with a dense branch over the numeric features. They report accuracy 0.9980, F1 0.9988, five-fold cross-validation, a learning-curve simulation of growing crowd data that plateaus near 99.7 % accuracy, and about 0.20 ms per URL.

The paper is clear about the gap and about its own limits: results come from one dataset, real-world validation is still needed, and the crowd channel can be abused. Two points limit what can be taken from it. The crowd loop is not run with real reports, and the page-content features it inherits from the dataset would require fetching every page in a live system, which the paper does not discuss.

B. Related Work

Supervised URL classifiers. As summarised in [1], Vajrobol et al. [8] rank features by mutual information and use logistic regression (99.97 % with five features on PhiUSIIL); Etem and Teke [9] reduce 51 features to two with t-SNE and report 99.7 % for a decision tree; Alnemari and Alshammari [10] find Random Forest best among classical models (97.3 %). Ma et al. [11] and Sahingoz et al. [12] are early and widely cited studies of detecting malicious sites from URL features. All of these train once on a fixed dataset.

Combined and deployed systems. Fan [13] combines SVM and Naive Bayes over text, visual and URL features (99.16 %). Priya et al. [14] build a Random Forest and logistic regression browser extension (97 % on 11,055 URLs) and Sawant et al. [15] integrate ML, DL and NLP in an extension; the base paper notes that both neglect latency and continuous retraining [1]. *Unsupervised detection.* Assefa and Katarya [16] use an autoencoder over 16 features (91.24 %), which suits novel sites but scores lower.

Crowds and humans in the loop. Huang et al. [17] show that crowdsourced cyber-threat intelligence can contain misleading information, and Lazaros et al. [18] review human-in-the-loop AI for high-stakes settings; both motivate verification before crowd data reaches a model. TABLE I compares the works discussed.

TABLE I
RELATED WORK (RESULTS AS REPORTED IN [1])

Work	Approach	Data	Reported result	Limitation
[1]	LSTM-attention + dense; crowd loop	PhiUSIIL, 235,804	Acc. 99.8 %	Crowd loop simulated
[8]	Mutual info + LR	PhiUSIIL	99.97 %	Static data
[9]	t-SNE + decision tree	51-feature set	99.7 %	One dataset
[10]	Classical models	UCI domains	97.3 % (RF)	Static data
[14]	RF / LR extension	11,055 URLs	97 %	Latency unstated
[16]	Autoencoder	Small set	91.24 %	Lower accuracy
[13]	SVM + NB ensemble	Text, visual, URL	99.16 %	Predefined features

Three observations follow from the survey. First, accuracy figures between 91 % and 99.97 % are reported on different, static datasets, so they cannot be ranked against each other, and none of them says how a model behaves on a campaign that starts after training. Second, the studies that use only the URL string trade some accuracy for speed and safety, because no page has to be fetched, which is the trade this project also makes. Third, the human-in-the-loop idea is well motivated but rarely built: verification protects the training data, yet it adds delay and needs staff, and a purely vote-based alternative is faster but easier to game. PhishGuard implements the vote-based variant and measures what it does, so that the trade-off can be discussed with evidence.

The gap is therefore a system that keeps its detector current from live community evidence, explains its verdicts, and reports measured behaviour of the loop. Blacklists fail on zero-day URLs, and few systems join lexical, structural and domain features with continuously updated threat feeds [1].

III. METHODOLOGY

A. Pipeline and Data

PhishGuard implements the three phases as one pipeline (Fig. 1). The training data is a balanced sample of 40,000 URLs (20,000 phishing, 20,000 legitimate, mean length 51.5 characters) of a public labelled set of about 420,000 URLs [19]; its licence was not verified. Rows are cleaned (missing values, duplicates, labels mapped to 0/1) and split 80/20, stratified, with a fixed seed: 32,000 for training and 8,000 for testing. Only 266 URLs carry a scheme, so each URL is normalised first (scheme added, host lower-cased, www. removed).

Cleaning follows a fixed recipe: rows with a missing URL or label are dropped, text labels (bad, phishing, malicious; good, benign, legitimate) are mapped to 1 and 0, whitespace is trimmed, URLs shorter than four characters are removed and duplicates are dropped. Because normalisation is deterministic and never raises on malformed input (for example an invalid port or an unmatched bracket), the same function serves training rows and live requests. Phishing URLs in the sample are slightly longer than legitimate ones (55.9 against 47.1 characters on average), but the length distributions overlap strongly, so no single simple feature separates the classes.

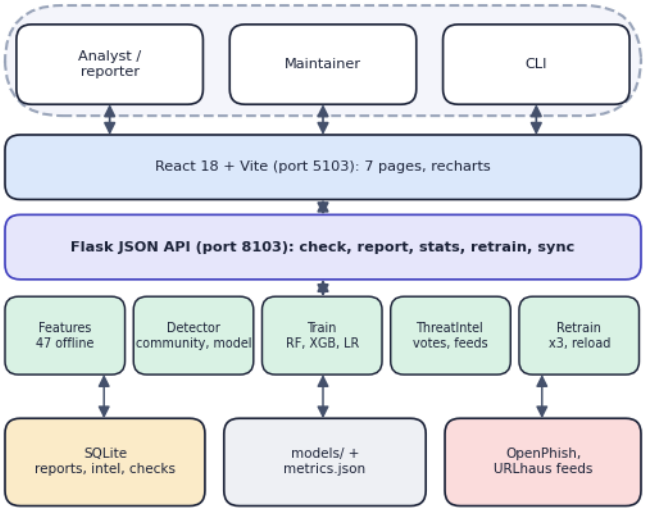


Fig. 1. System architecture of PhishGuard.

B. Features

Forty-seven features are computed from the URL string alone, with no DNS, WHOIS or page download, so training and run time share one code path. Twenty-three lexical features count lengths, separators, digits and special characters, the Shannon entropy of the URL, credential-related words and encoded characters. Nine structural features describe the build of the URL: IP host, port, sub-domain count, path depth, shorteners and executable extensions. Fifteen domain features describe the registered domain and its TLD: entropy, digit and hyphen counts, vowel ratio, TLD class, brand names inside or outside the domain, and membership in a built-in list of 145 popular domains used as a reputation signal. An explainer turns the features into red flags such as “host is a raw IP address”.

$$H(s) = -\Sigma_c\, p(c)\, \log_2 p(c) \qquad (1)$$

C. Classifiers and Selection

Three models are fitted on the same features and split: Logistic Regression with scaling (C = 1), Random Forest [20] with 300 trees and XGBoost [4] with 400 trees, depth 8 and learning rate 0.08. They represent a linear model, bagged trees and boosted trees, and train in seconds. The model with the highest F1 on the test set is saved and loaded by the detector. No hyper-parameter search was run. We did not reproduce the base paper's LSTM network: it relies on text and page features of a much larger dataset, whereas tree ensembles on engineered features are light and can explain their decisions.

D. Community Intelligence and Decision Rule

Users report a URL as phishing or safe with a reporter name and a note. With a phishing votes and b safe votes, $n = a - b$; a URL is confirmed phishing if $n \geq 2$ or if it comes from a trusted public feed (OpenPhish, URLhaus), confirmed safe if $n \leq -2$, and pending otherwise. At check time a confirmed entry for the URL or its host decides at once. Otherwise the model probability p is adjusted and thresholded:

$$p' = clip(r \cdot p + 0.15 \cdot n,\ 0,\ 1) \qquad (2)$$

where r = 0.4 if the registered domain is popular and its TLD is common or longer than two letters, and 1 otherwise. The verdict is phishing if $p' \geq 0.60$, suspicious if $p' \geq 0.35$ and safe otherwise (Fig. 2). Confirmed URLs are repeated three times in the training matrix and all models are retrained; the detector then reloads the best model under a lock, without a restart.

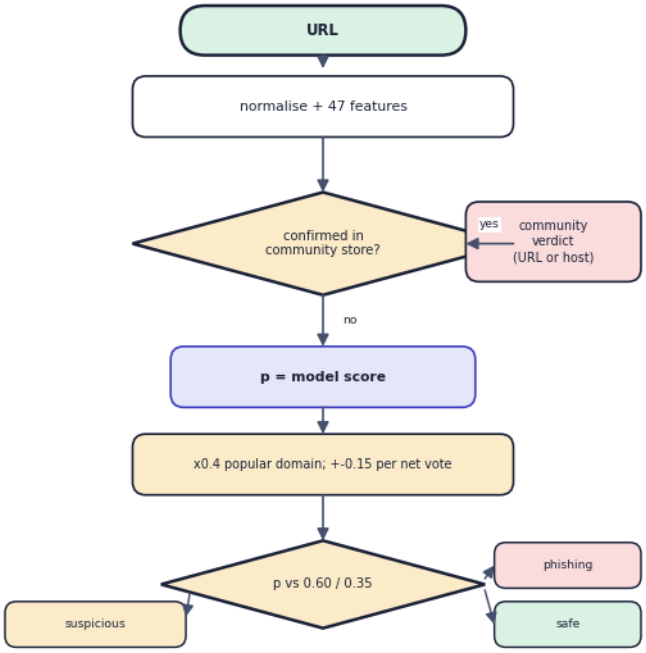


Fig. 2. Decision order of the detector for one URL.

E. Implementation

A Flask application exposes the JSON API (check, report, stats, retrain, sync and five read-only endpoints) on port 8103; the React interface on port 5103 proxies /api to it. State is a SQLite file with three tables (reports, intel, checks) and the model files. The interface has seven pages: landing, check (single and batch), report, threat intelligence, models and activity. SQL statements use bound parameters and the URL is never fetched, so a malicious link cannot run anything on the server.

F. Robustness and Security

Input is untrusted, so the system never opens the URL; it only reads the string. An empty or whitespace-only URL, an unknown verdict and an unknown status filter return HTTP 400 with a JSON error, and unusual strings (a bare domain, a 5,000-character URL, a unicode host, a percent-encoded traversal, javascript: and data: URLs) are classified as text without error. Reports and notes are stored through bound parameters, and the web pages escape their output, so SQL and script injection are neutralised. A failing public feed is reported per feed and never stops the application. Known weaknesses are listed in Section IV-E.

IV. RESULTS

A. Test Conditions

Models were evaluated on the 8,000 held-out URLs (4,000 per class) that no model saw in training; re-scoring the saved models reproduced the stored metrics exactly. We also ran five-fold stratified cross-validation over all 40,000 URLs. Timings were measured in process with Flask's test client on the machine above. Results are therefore from one sampled dataset and are not comparable with other datasets.

TABLE II
HELD-OUT TEST RESULTS

Model	Acc. %	Prec. %	Rec. %	F1 %	AUC
XGBoost	92.93	93.53	92.23	92.88	0.9809
Random Forest	92.49	92.69	92.25	92.47	0.9776
Logistic Regr.	82.81	86.94	77.22	81.80	0.9218
XGBoost, 5-fold CV	92.59	92.91	92.21	92.56	0.9792

B. Model Performance

XGBoost is selected: accuracy 92.93 %, F1 92.88 % and ROC-AUC 0.981 (TABLE II, Fig. 3). Random Forest is within 0.5 points; Logistic Regression is clearly behind (recall 77.2 %), so the classes are not linearly separable in these features. Cross-validated F1 is 92.56 % with a standard deviation of 0.18 points. XGBoost makes 255 false positives and 311 false negatives on the test set. Lexical features alone reach F1 89.2 %, domain features alone 77.7 % and structural features alone 66.0 %; all 47 give 92.8 %, and removing the two strongest features (is_common_tld, is_popular_domain) costs only 0.2 points. F1 is flat between thresholds 0.40 and 0.60 (92.9 % to 92.4 %), while lowering the threshold to 0.35 raises recall to 94.4 % at a false-positive rate of 9.6 %. With the deployed thresholds, 3,583 of 4,000 phishing URLs are called phishing and 187 suspicious, while 188 of 4,000 legitimate URLs are called phishing.

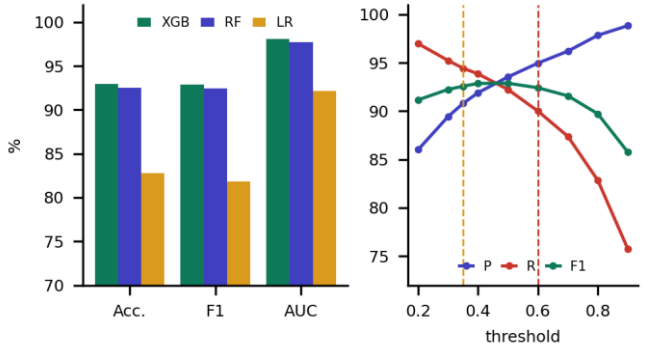


Fig. 3. Test metrics (left) and precision, recall and F1 against the threshold (right).

TABLE III
FEATURE ABLATION (XGBOOST REFITTED, THRESHOLD 0.5)

Features used (count)	Acc. %	F1 %	AUC
Lexical only (23)	89.3	89.2	0.961
Structural only (9)	64.9	66.0	0.707
Domain only (15)	78.7	77.7	0.883
Lexical + domain (38)	92.5	92.4	0.979
All features (47)	92.8	92.8	0.981
Without two strongest (45)	92.7	92.6	0.981

The ablation (TABLE III) shows where the signal comes from. The most important features of the selected model are is_common_tld (0.195), is_popular_domain (0.165) and suspicious_word_count (0.121), followed by brand_outside_domain and has_executable_extension (Fig. 4, left). Because two of the top three are domain-reputation signals, they were removed together: F1 fell by only 0.2 points, so the model does not depend on them alone. The learning curve (Fig. 4, right) rises from F1 89.5 % with 3,200 training URLs to 93.2 % with 32,000 and is still rising, which suggests that more data would help; the final point differs from the shipped model by about 0.3 points because the rows were permuted for this experiment.

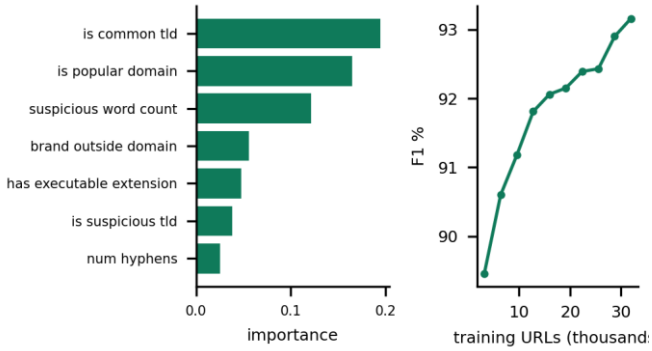


Fig. 4. XGBoost feature importance (left) and learning curve (right).

C. Crowd Loop and Output Screens

The crowd loop was exercised on the running application (Fig. 5). A look-alike document-share URL that the model scored at 9.5 % became phishing at 97.0 % after two community reports, decided by the community, with no retraining; two legitimate pages the model flagged (docs.python.org and amazon.in) were cleared by two safe reports each. Synchronising the OpenPhish and URLhaus feeds added 800 URLs, and a retrain from the Models page (Fig. 6) produced model version 2 from 40,000 base rows plus 807 confirmed URLs (XGBoost F1 92.99 % on a different test set of 8,162 URLs, so not a measured improvement).

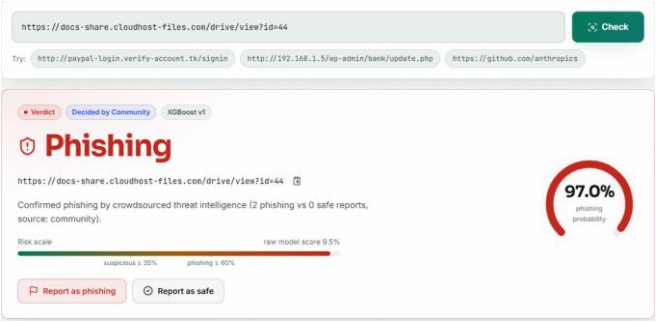


Fig. 5. Check screen: community verdict overrides the model.

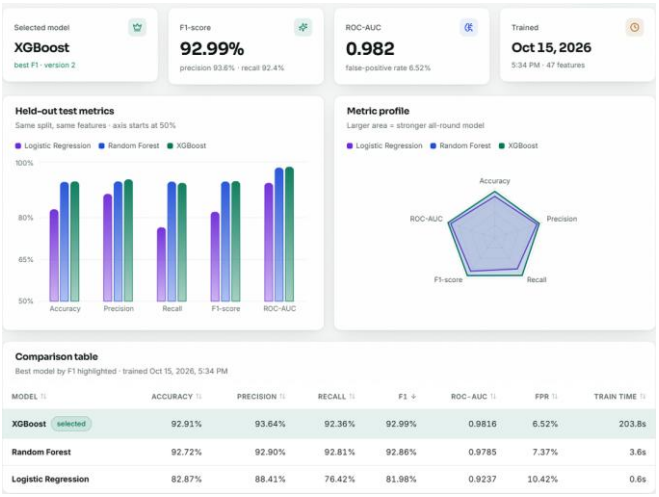


Fig. 6. Models screen after retraining (version 2).

To measure whether retraining improves the model, 250 to 2,000 correctly labelled test URLs were added to the training data (×3) and the model was scored on the remaining URLs: F1 changed by less than 0.1 point at every size (92.82 % to 92.76 % with 250; 92.83 % to 92.90 % with 2,000). Random in-distribution URLs therefore do not change the average score; the value of the loop is the immediate override and the reversal of known mistakes. Its benefit on a new campaign could not be measured with a static dataset.

TABLE IV
EFFECT OF ADDING CONFIRMED URLS TO TRAINING (MEAN OF 3 DRAWS)

Confirmed URLs added	Held-out URLs	F1 before %	F1 after %	Missed before	Missed after
250	7,750	92.82	92.76	305	305
500	7,500	92.85	92.81	295	297
1,000	7,000	92.79	92.72	277	278
2,000	6,000	92.83	92.90	238	235

TABLE IV gives the full experiment. The number of phishing URLs missed on the unseen part moves by two or three at most, and in one case it rises slightly, which is within the variation of a boosted-tree fit. This is the expected outcome when the model already saw 32,000 similar examples: correctly labelled URLs from the same distribution add little. The result should not be read as showing that crowd data is useless; it shows that, on this static dataset, the benefit of a crowd must come from information the model lacks, such as a new campaign or a mistake that a human can correct. That is what the override provides, and measuring the adaptation to a genuinely new distribution is the main open experiment of this work.

D. Speed, Tests and Comparison

Feature extraction takes 0.08 ms per URL and a check through the API takes 8.0 ms on average (p95 9.7 ms). Scoring 8,000 feature rows in one batch takes a few milliseconds, and featurising all 40,000 training URLs takes about four seconds. Training the three models takes 15.1 s in total (Random Forest 12.2 s, XGBoost 2.7 s, Logistic Regression 0.2 s), so a retrain on the base data is quick; the retrain in the live run is slower because it also featurises and splits the enlarged data and ran while a browser polled the page. All 95 automated checks pass: 3 original tests, a 20-step API smoke test and 72 extended checks covering unit, integration, functional, validation, security and performance cases, including SQL-injection and script-tag inputs. The base paper reports accuracy 99.80 % and F1 0.9988 [1]. Our 92.93 % is 6.9 points lower, but the datasets, features and models differ: it used 235,804 URLs, page-content features and an LSTM network, whereas we use a 40,000-URL sample, URL-only features and tree ensembles. No claim of matching it is made.

E. Failure Cases and Limits

The most confident false negatives are ordinary-looking pages (a blog post, a form-sharing page, a PDF on a research site): a phishing page on a clean host has no suspicious URL features. False positives are unusual pages on small sites and, in the live set, docs.python.org (0.60) and amazon.in (0.79), because the hand-written popular-domain list lacks python and gives no discount to two-letter country TLDs. A confirmed phishing URL marks every URL on its host as phishing, which would over-block shared hosting. The application has no sign-in, votes are anonymous, and no user-acceptance, load or independent security test was done.

V. CONCLUSION AND FUTURE SCOPE

PhishGuard implements the three-stage design of the base paper end to end: 47 offline URL features, XGBoost as the best of three classifiers (F1 92.9 %, ROC-AUC 0.981, five-fold F1 92.6 %), a community store whose confirmed verdicts override the model in both directions, and a retraining loop with hot reload that ran on real feed data. Against the project objectives all six were met, except that crowd confirmation is a vote count and not a human review. The key finding is that the loop's value is the immediate override, not a better average score. Limits are the small single-source dataset, URL-only features, anonymous votes and the absence of user testing.

Several practical lessons come out of the work. Interpretable URL features and a reason sentence make a verdict easy to question, and the community channel turned a false negative and two false positives into correct verdicts within one check. The weakest points are all on the trust side: anonymous votes, no sign-in, host-wide override and a hand-written reputation list. They are cheap to describe and moderate to fix, and the next version should treat them before any claim about deployment is made.

Future work: evaluate on larger and more recent data and on URLs collected after training; add sign-in, rate limiting and an administrator review queue; replace the hand-written reputation list; study the loop under noisy reports and new campaigns; and reproduce the base paper's network for a like-for-like comparison.

REFERENCES

[1] L. Aldhuwayhi and A. Alromih, "A dynamic phishing URL detection system integrating advanced machine learning and crowdsourced threat intelligence," IEEE Access, vol. 14, pp. 88393-88404, 2026, doi: 10.1109/ACCESS.2026.3700833.

[2] K. Krombholz, H. Hobel, M. Huber and E. Weippl, "Advanced social engineering attacks," Journal of Information Security and Applications, vol. 22, pp. 113-122, Jun. 2015, doi: 10.1016/j.jisa.2014.09.005.

[3] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

[4] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 785-794.

[5] Pallets Projects, "Flask documentation." [Online]. Available: https://flask.palletsprojects.com

[6] Meta Platforms, "React documentation." [Online]. Available: https://react.dev

[7] A. Prasad and S. Chandra, "PhiUSIIL phishing URL (website)," UCI Machine Learning Repository, 2024, doi: 10.1016/j.cose.2023.103545.

[8] V. Vajrobol, B. B. Gupta and A. Gaurav, "Mutual information based logistic regression for phishing URL detection," Cyber Security and Applications, vol. 2, p. 100044, Jan. 2024, doi: 10.1016/j.csa.2024.100044.

[9] T. Etem and M. Teke, "Advanced phishing detection: Leveraging t-SNE feature extraction and machine learning on a comprehensive URL dataset," Acta Infologica, vol. 231, no. 221, pp. 213-221, 2024, doi: 10.26650/acin.1521835.

[10] S. Alnemari and M. Alshammari, "Detecting phishing domains using machine learning," Applied Sciences, vol. 13, no. 8, p. 4649, Apr. 2023, doi: 10.3390/app13084649.

[11] J. Ma, L. K. Saul, S. Savage and G. M. Voelker, "Beyond blacklists: Learning to detect malicious web sites from suspicious URLs," in Proc. 15th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2009, pp. 1245-1254.

[12] O. K. Sahingoz, E. Buber, O. Demir and B. Diri, "Machine learning based phishing detection from URLs," Expert Systems with Applications, vol. 117, pp. 345-357, 2019.

[13] Z. Fan, "Detecting and classifying phishing websites by machine learning," in Proc. 3rd Int. Conf. Applied Machine Learning (ICAML), Jul. 2021, pp. 48-51, doi: 10.1109/ICAML54311.2021.00018.

[14] K. S. Priya, J. Chandrika and M. P. Lakshmi, "Machine learning-based phishing website detection: a comprehensive approach for cyber security," in Proc. 5th Int. Conf. Recent Trends in Computer Science and Technology (ICRTCST), Apr. 2024, pp. 344-349, doi: 10.1109/ICRTCST61793.2024.10578472.

[15] S. Sawant, R. Savakhande, O. Sankhe and S. Tamboli, "Phishing detection by integrating machine learning and deep learning," in Proc. 11th Int. Conf. Computing for Sustainable Global Development (INDIACom), Feb. 2024, pp. 1078-1083, doi: 10.23919/INDIACom61295.2024.10499100.

[16] A. Assefa and R. Katarya, "Intelligent phishing website detection using deep learning," in Proc. 8th Int. Conf. Advanced Computing and Communication Systems (ICACCS), vol. 1, Mar. 2022, pp. 1741-1745, doi: 10.1109/ICACCS54159.2022.9785003.

[17] H. Huang, N. Sun, M. Tani, Y. Zhang, J. Jiang and S. Jha, "Unmasking disinformation: Enhancing cyber threat intelligence through crowdsourced analysis and AI-driven training," in Proc. 59th Hawaii Int. Conf. System Sciences (HICSS), 2026, pp. 6049-6058, doi: 10.24251/HICSS.2026.743.

[18] K. Lazaros, A. G. Vrahatis and S. Kotsiantis, "Human-in-the-loop artificial intelligence: A systematic review of concepts, methods, and applications," Entropy, vol. 28, no. 4, p. 377, Mar. 2026, doi: 10.3390/e28040377.

[19] "Using machine learning to detect malicious URLs," public dataset repository (data/data.csv). [Online]. Available: https://github.com/faizann24/Using-machine-learning-to-detect-malicious-URLs

[20] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5-32, 2001.